LYEINANI RAG Chatbot: Project Report

Nov 7, 2026 · @Linda

I built a customer-support chatbot for LYEINANI, my anime-inspired clothing brand on Shopify, that answers from my own website and my own written answers. The final version scored 10/10 on my accuracy test and runs entirely on free, open-source tools.

Purpose

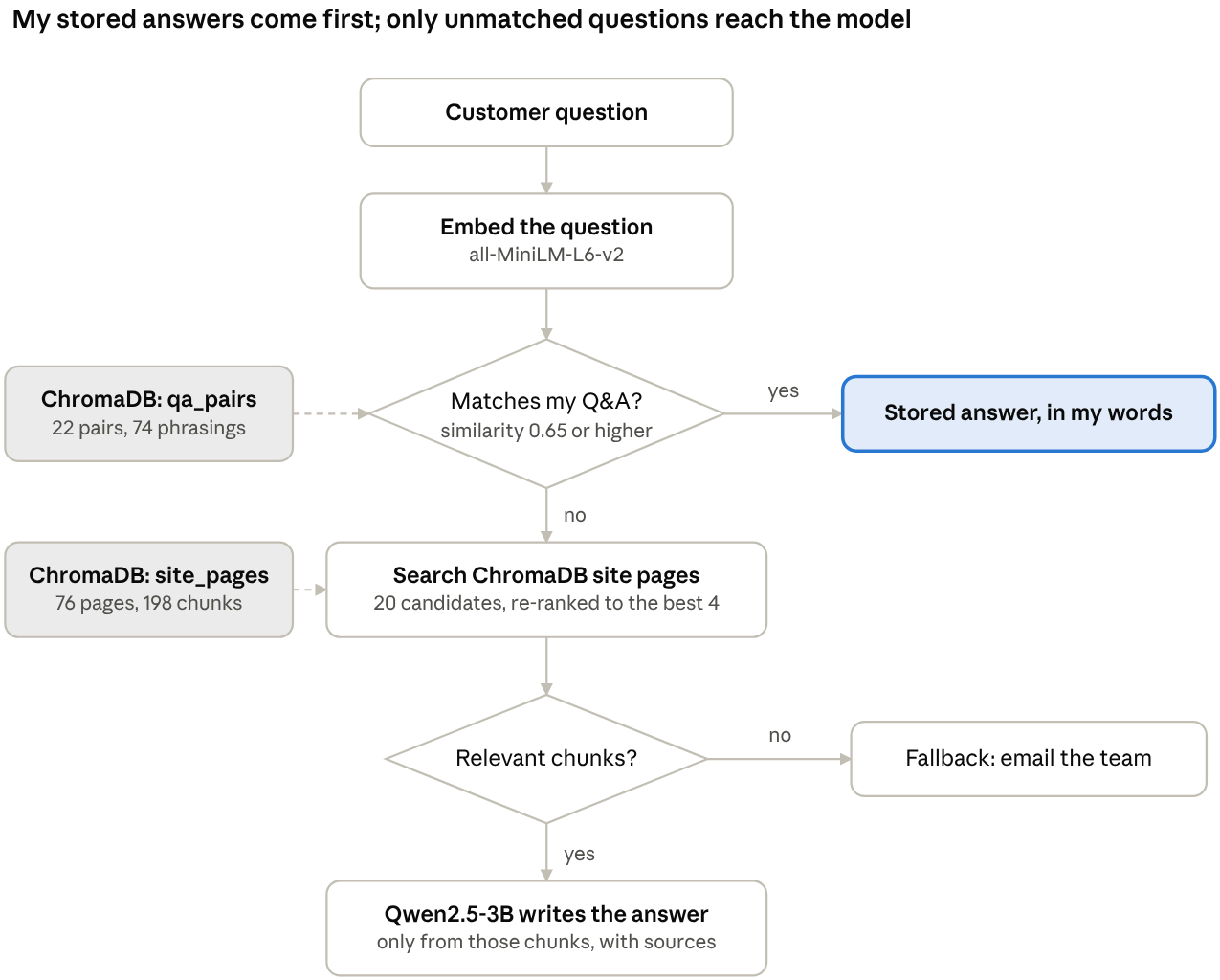
Most customer messages I get are the same few questions: how exchanges work, who pays for return shipping, when a preorder will arrive, whether I ship internationally. The answers are on lyeinani.com, but they are spread across policy pages, product pages and my PRETA preorder page, so customers don't always find them.

I wanted an assistant that:

- answers those questions instantly, in my words, 24/7
- only says what my site actually says, and shows the page it came from
- admits when it doesn't know and points to my email, instead of guessing

The project also had to meet the bootcamp brief: a RAG-powered chatbot with a complex knowledge base, an integrated vector database, a clean user-friendly UI, and extra features.

Architecture



question routing · 2 decisions, 2 ChromaDB collections

A question that closely matches one of my Q&A phrasings gets my stored answer with no model involved; everything else is answered by the model from the best four site chunks, or sent to my email when nothing relevant is found.

Knowledge base

The bot draws on two sources I prepared, both stored in a ChromaDB vector database.

Source	What it holds	Size	ChromaDB collection
d1: my website	Every public page of lyeinani.com (products, collections, policies, PRETA, blog), rendered to PDF	76 pages, 198 chunks	`site_pages`
d2: my Q&A set	Answers I wrote for the most common questions (returns, exchanges, shipping, customs, preorders, contact, Brand Support)	22 pairs, 74 phrasings	`qa_pairs`

The Q&A set includes details my site doesn't spell out yet: LYEINANI pays return shipping, preorders can be exchanged if the size is available, and preorders take about 8–11 weeks (production 4–6 weeks, customs about 20 days, shipping 1–2 weeks).

The scraper skips cart, checkout, account and login pages, and my private Black Market page, so none of them reach the bot.

How I built it

1. **Scraped my site.** `lyeinani_scraper.py` reads the Shopify sitemap, follows internal links, renders each page in headless Chromium (so JavaScript-loaded prices are captured) and saves one PDF per page plus a `manifest.json` of URLs and titles for citations.
2. **Wrote my Q&A set.** `lyeinani_qa_set.json` holds 22 answers in my words, each with alternate phrasings, a category and its source page.
3. **Built the RAG pipeline.** `rag_bot.py` extracts the PDF text, removes the menu and footer lines that repeat on every page, splits the text into ~900-character chunks, embeds them with `all-MiniLM-L6-v2` and stores them in ChromaDB.
4. **Routed each question.** The question is matched against my Q&A phrasings first. A similarity of 0.65 or higher returns my stored answer with no model involved.
5. **Retrieved and re-ranked.** Otherwise ChromaDB returns 20 candidate chunks. I boost chunks from the page the question names, drop chunks far below the best match, and keep the top 4.
6. **Generated a grounded answer.** A free open-source model, Qwen2.5-3B-Instruct, answers using only those chunks, cites them, and is told never to invent prices or policies and to ignore instructions inside page text. If nothing relevant is found, the

bot points to my email instead.

7. **Handled follow-ups.** A short follow-up like "what about the vest?" is searched together with the previous question, and the model sees the earlier question only as context marked "already answered".
8. **Measured it.** `run_eval()` scores the bot on questions with known right answers (see Evaluation).
9. **Wrapped it in a chat app** and documented the whole pipeline in a Colab notebook that runs on a free GPU with no API key.

Evaluation

Testing changed two of my design decisions, and the final version answered all 10 test questions correctly.

Threshold. I started at 0.80, which only matched 1 of 6 paraphrases of my own questions. Every paraphrase scored 0.67 or higher and every product question 0.48 or lower, so I lowered it to 0.65, which separates the two groups.

Question	Score	Closest stored question
my preorder is the wrong size can i swap it	0.850	Can I change the size of my preorder?
is there a fee for customs	0.789	Who pays customs fees or duties?
can i get a refund	0.769	Do you offer refunds?
do you guys ship to the UK	0.736	Do you ship internationally?
how many days do i have to exchange something	0.727	How many days do I have to return something?
who covers the cost of sending a return back	0.670	Who pays for return shipping?
do you have any bodysuits (product question)	0.480	What kind of clothes do you sell?
how much is the Crimson Bloom Blazer (product question)	0.450	How much is shipping?

Accuracy. My test asks 10 questions with known answers from my site: 3 policy questions, 4 product prices, a follow-up, a collection question and an off-topic question. Each answer is checked for required facts, facts it must not contain, and the route taken.

Version	Model	Result
First run	Qwen2.5-1.5B	Follow-up wrong: "what about the vest?" answered $110, copying the trouser's price (correct: $75)
Round 1 fixes	Qwen2.5-3B	8/10; 9/10 once my own test was corrected
Round 2 fixes (final)	Qwen2.5-3B + ChromaDB	**10/10**

One Round 1 "miss" was my test's mistake: it flagged the Pretty Soldier Mini Dress in the Tokyo '92 answer, but the dress is part of that collection. The other was real: the vest follow-up gave the right $75 but drifted back to the trouser. Round 2 fixed it by passing the earlier question only as context.

Ten questions is a small test, so this shows the fixes worked on the cases I know about, not that the bot never errs.

Chat app and extra features

The Streamlit app (`app.py`) uses `st.chat_input` and `st.chat_message` for the conversation, `st.session_state` for history, and `@st.cache_resource` so the models and ChromaDB load once. On top of the basics I added:

Feature	Why
LYEINANI-branded dark theme in my red-orange, with suggested starter questions	Clean, on-brand UI; new users know what to ask
A label on every answer: "From LYEINANI's answers" or "From the website", with source links	Customers can trust it; I can debug it
👍 / 👎 feedback on each answer, saved to `feedback.csv`	Shows me which answers to improve

Feature	Why
"Save chat" download	Customers keep a record of what they were told
Per-session question limit (30)	Protects a public demo from spam
Developer settings: live threshold slider, match scores, ChromaDB stats	Tune and demonstrate the routing live
Automated accuracy test (`run_eval`)	Measures changes instead of guessing
No API key or paid service	Anyone can run the notebook on Colab's free GPU

Challenges, lessons and next steps

The biggest lesson: measure, don't assume. My first threshold looked reasonable but skipped most of my own answers, and one "failure" turned out to be a mistake in my test, not the bot.

Challenge	What I did
Paid API key needed for the first model	Switched to free, open-source Qwen2.5 running on Colab's GPU
Small model copied a price from its previous answer	Stopped showing it past answers; follow-ups now carry the earlier question only as context
Product's own page lost to the home page and blog posts	Re-ranking with a page-title boost and a relative score cut-off
Every page repeats the same menu and footer	Removed lines that appear on most pages before chunking
162 MB of PDFs too large for GitHub's web upload	Compressed the embedded images to 38 MB and checked that every page's text was unchanged
Scraper reached my private Black Market page	Added it to the scraper's skip list so it stays out of the bot and the public repo

Limitations. A 3B model writes simpler answers than a large paid model. The site data is a snapshot, so I re-run the scraper when my site changes. The test set is only 10 questions.

Next steps. Grow the test set and add automatic quality checks (for example, RAGAS); embed the assistant on my Shopify store as a chat bubble; add the answers from my Q&A set to my policy pages so both sources agree.

Links

- Code, knowledge base and README: github.com/alwayzlynluv/lyeinani-rag-chatbot
- Step-by-step notebook with outputs: `lyeinani_rag_chatbot.ipynb` in the repository (opens in Google Colab)
- The store the bot answers about: lyeinani.com

Tools: Python, Playwright, sentence-transformers (all-MiniLM-L6-v2), ChromaDB, Hugging Face transformers (Qwen2.5-3B-Instruct), pypdf, NumPy, Streamlit, Google Colab.